



ARTIFICIAL NEURAL NETWORKS (ANN)

LAB 07

Ms. Shakeela Shaheen

LAB 07: TRANSFER LEARNING AND USING PRE-TRAINED CNN MODELS

Learning Objectives

By the end of this lab, students will:

1. Understand what transfer learning is.
2. Understand why pretrained models are powerful.
3. Use pretrained CNN models (e.g., MobileNetV2, ResNet).
4. Apply padding and understand its importance.
5. Apply data augmentation.
6. Freeze and unfreeze layers.
7. Perform fine-tuning.
8. Compare training from scratch vs transfer learning.

PART 1 – Why Transfer Learning?

Problem with Training CNN From Scratch

Deep networks like ResNet:

- Millions of parameters
- Require huge datasets
- Require powerful GPUs
- Long training time

Example:

- ImageNet has 1.4 million images.
- Most student datasets have < 10,000 images.

Training from scratch → overfitting.

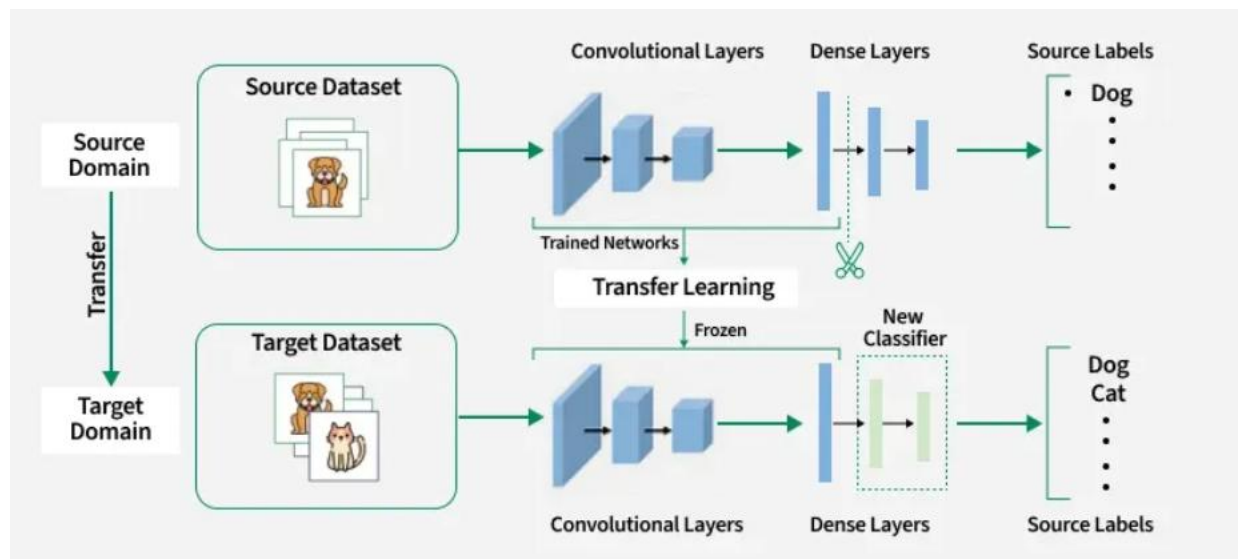
What is Transfer Learning?

- Transfer learning means: Train your model on one dataset and run it on another
- Use knowledge learned on one large dataset and transfer it to a new task.

- Transfer learning is a machine learning technique where a model trained on one task is repurposed as the foundation for a second task. This approach is beneficial when the second task is related to the first or when data for the second task is limited.

Example:

- Pretrained on ImageNet (1.4 million images)
- Already knows edges, textures, shapes
- We reuse those learned filters



Intuition

Think of a student:

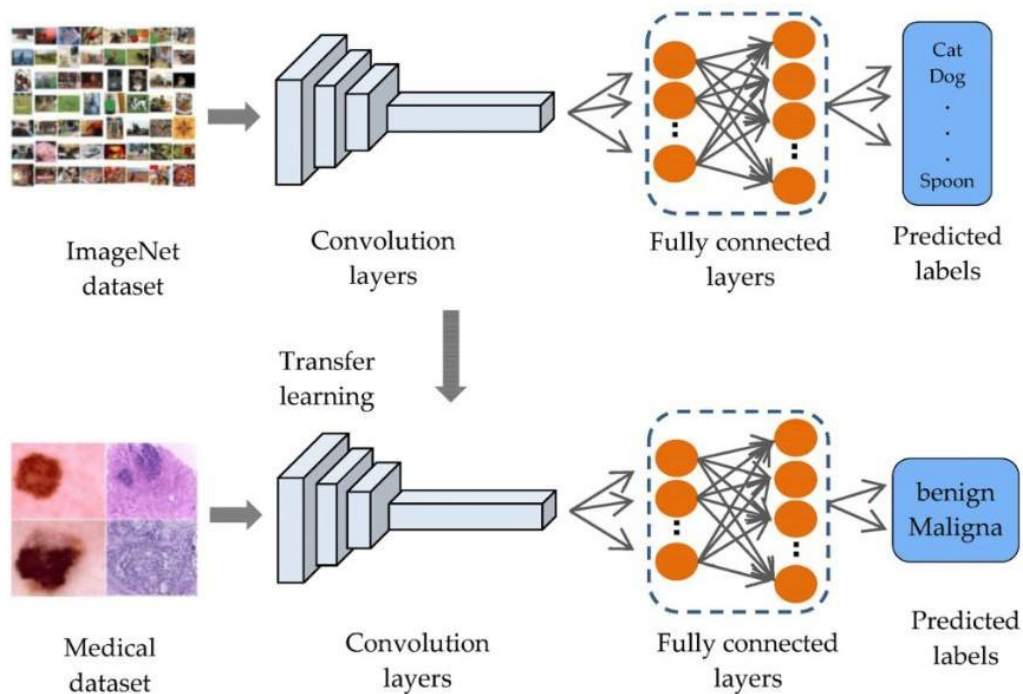
- Already knows basic math
- Learning physics becomes easier

Same idea.

It has become a common and highly successful approach to re-use pre-trained networks for new task:

1. Chop off output layer or last n layers.
2. Add new layers on top.
3. Train weights of new layers to solve new tasks.

The pre-trained weights can be fixed or also re-trained for the new tasks.



Note:

Transfer learning and fine-tuning both use pre-trained models but differ in scope: transfer learning usually freezes all pre-trained layers to train only a new classifier head, while fine-tuning unfreezes some or all layers to retrain weights on new, specific data. Transfer learning is faster, while fine-tuning offers higher domain accuracy.

- **Use Transfer Learning (Feature Extraction):** When the new dataset is small, or very similar to the original pre-training data.
- **Use Fine-Tuning:** When the new dataset is large and different from the original, requiring deeper adaptation.

PART 2 – What is a Pre-trained Model (A type of transfer learning)?

A pre-trained model is a machine learning model that has been previously trained on a large dataset for a specific task (usually general-purpose) and can then be reused or fine-tuned for a different but related task. Pre-trained models save development teams time, data and computational resources compared to training a model from scratch.

Popular pre-trained CNNs:

- MobileNetV2
- ResNet50
- VGG16

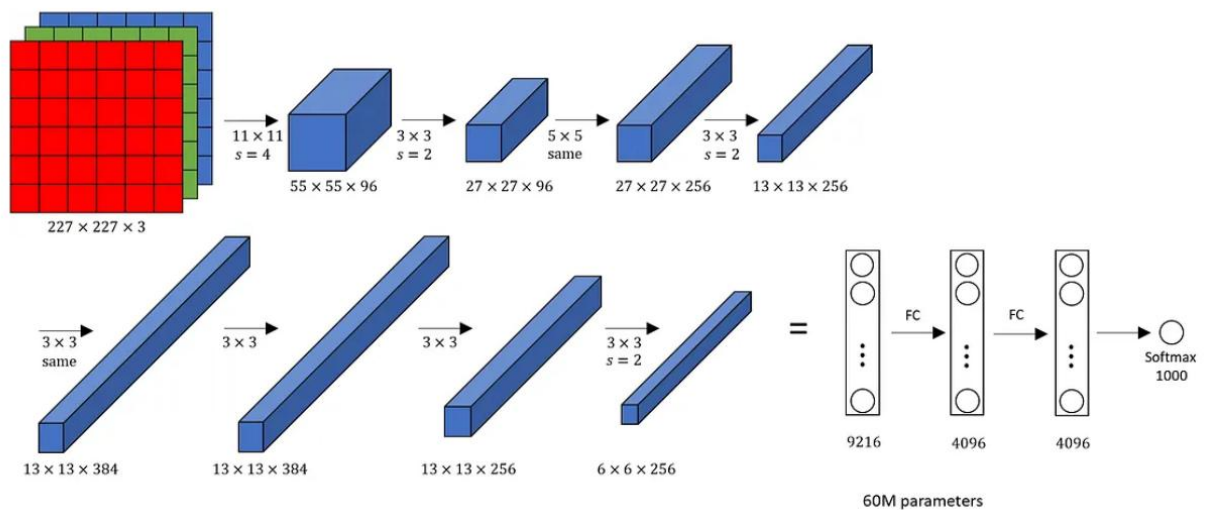
- EfficientNet

All trained on ImageNet.

These models have learned:

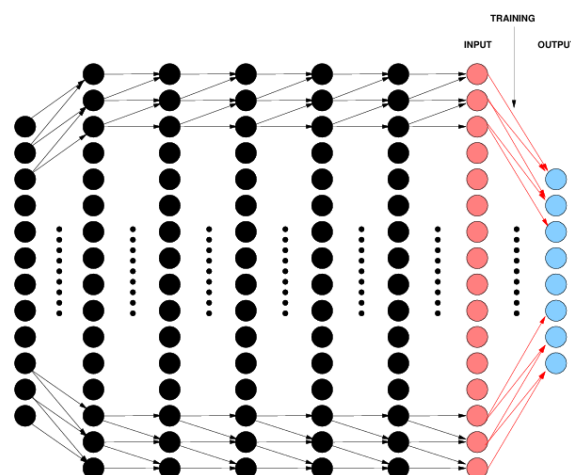
- Edge detectors (early layers)
- Shape detectors (middle layers)
- Object detectors (deep layers)

AlexNet Architecture



2010 → ML model → 28%
 2011 → ML Model → 25%
 2012 → AlexNet → 16.4%
 2013 → ZFNet → 11.7%
 2014 → VGG → 7.3%
 2015 → GoogleNet → 6.7%

2016 → ResNet → 3.5



PART 3– Data Augmentation

Why Data Augmentation?

If dataset is small:

- Model overfits.
- Learns specific patterns.
- Fails to generalize.

Data augmentation artificially increases dataset size.

Common Augmentations

- Rotation
- Flipping
- Zoom
- Shift
- Brightness
- Horizontal flip

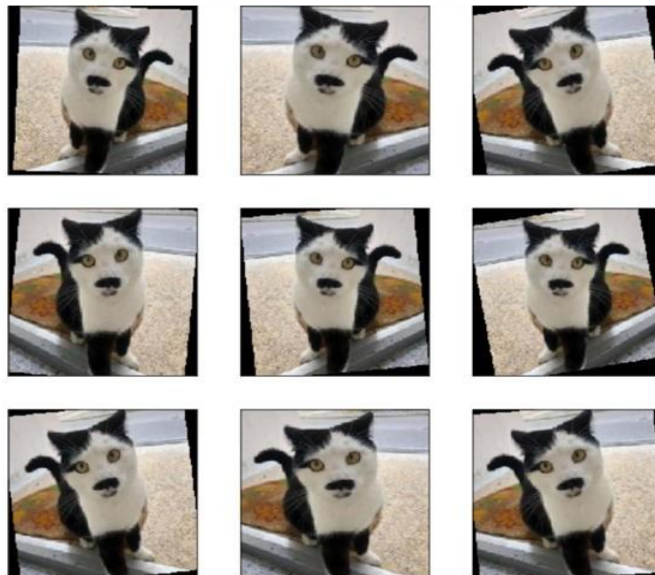
Why It Works?

Data Augmentation: Example

Original image:



Augmented images:



It teaches model:

“Object is still a cat even if rotated.”

Improves generalization.

PART 4 – Padding in CNN

What is Padding?

Padding adds extra pixels (usually zeros) around image borders.

Example:

Original $5 \times 5 \rightarrow$ After padding $\rightarrow 7 \times 7$

Why Padding?

Without padding:

- Image shrinks after convolution.
- Border information is lost.

With padding:

- Preserve spatial dimensions.
- Keep edge information.
- Padding is the best approach, where the number of pixels needed for the convolutional kernel to process the edge pixels are added onto the outside copying the pixels from the edge of the image.
- Fix the Border Effect Problem With Padding.

Types of Padding

Valid Padding $\rightarrow$ No padding

Same Padding $\rightarrow$ Output size = input size

PART 5 – Practical Implementation

We will use CIFAR-10 (resized to 96×96 or 128×128).

Step 1 – Load Dataset

```
from tensorflow.keras.datasets import cifar10
import tensorflow as tf
import numpy as np

(X_train, y_train), (X_test, y_test) = cifar10.load_data()
```

Step 2 – Resize Images (Important for Pretrained Models)

Pretrained models expect larger input sizes.

```
X_train = tf.image.resize(X_train, (96,96))
X_test = tf.image.resize(X_test, (96,96))

X_train = X_train / 255.0
X_test = X_test / 255.0
```

Step 3 – Data Augmentation Layer

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.1),
    tf.keras.layers.RandomZoom(0.1)
])
```

Step 4 – Load Pretrained Model

Use MobileNetV2:

```
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(96,96,3),
    include_top=False,
    weights='imagenet'
)
```


Step 5 – Freeze Base Model

```
base_model.trainable = False
```

We don't want to destroy pretrained weights initially.

Step 6 – Build Transfer Learning Model

```
from tensorflow.keras import layers, models

model = models.Sequential([
    data_augmentation,
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])
```

Step 7 – Compile Model

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Step 8 – Train Model

```
history = model.fit(
    X_train, y_train,
    epochs=10,
    validation_split=0.2
)
```

PART 6 – Fine-Tuning

Now unfreeze some layers.

Now unfreeze some layers.

Python

```
base_model.trainable = True

for layer in base_model.layers[:-20]:
    layer.trainable = False
```

Compile with lower learning rate:

Python

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

LAB TASK

Scenario

You are hired as a **Junior AI Engineer** at an aerial inspection company.

The company uses drones (UAVs) to inspect:

- Bridges
- Buildings
- Transmission towers

They collect images of concrete structures.

Your task:

Build an image classification system that detects whether a structure image shows:

- Crack Present
- No Crack

Real-World Constraint

- Dataset is small (1,200 images total)
- Images captured in different lighting conditions
- Some images rotated
- Some zoomed
- Some partially blurred

Management tried training CNN from scratch → overfitting badly.

Your job:

Use Transfer Learning to build a robust model.

You must:

1. Apply transfer learning using a pretrained CNN
2. Apply data augmentation
3. Implement padding correctly
4. Freeze and fine-tune layers properly
5. Compare:

- From scratch CNN
- Frozen transfer learning
- Fine-tuned transfer learning

6. Analyze overfitting behavior

Dataset

You may use:

- Concrete Crack Dataset (public)
OR
- CIFAR-10 (simulate crack vs no crack by selecting 2 classes)
OR
- Custom small dataset

Part 1 – Baseline Model (Failure Demonstration)

Task 1:

Build a CNN from scratch:

- Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool
- Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool
- Dense $\rightarrow$ Softmax

Train for 15 epochs.

Students must record:

- Training accuracy
- Validation accuracy
- Overfitting gap

Reflection:

- Why is it overfitting?
- What role does dataset size play?

Part 2 – Transfer Learning (Feature Extraction)

Step 1 – Load Pretrained Model

Use:

- MobileNetV2 OR ResNet50

```
base_model = tf.keras.applications.MobileNetV2(  
    input_shape=(128,128,3),  
    include_top=False,  
    weights='imagenet'  
)
```

Step 2 – Freeze Base

```
base_model.trainable = False
```

Explain in report:

- What freezing does mathematically
- Why gradient does not update base layers

Step 3 – Add Custom Head

Add:

- GlobalAveragePooling
- Dense(128, ReLU)
- Dropout(0.5)
- Dense(1, sigmoid)

Train for 10–15 epochs.

Students Must Compare:

Model	Train Acc	Val Acc	Overfitting?
CNN Scratch			
Transfer (Frozen)			

Part 3 – Data Augmentation Experiment

Add:

- RandomRotation
- RandomZoom
- RandomFlip
- RandomContrast

Train again.

Students must:

- Compare validation improvement
- Explain why augmentation helps

Part 4 – Fine-Tuning

Unfreeze last 20 layers:

```
base_model.trainable = True
for layer in base_model.layers[:-20]:
    layer.trainable = False
```

Reduce learning rate:

```
optimizer = tf.keras.optimizers.Adam(1e-5)
```

Train 5 more epochs.

Students Must Answer:

- Why reduce learning rate?
- What happens if we don't?
- Did validation improve?

Part 5 – Padding Analysis

Students must:

Build a small CNN with:

- padding='valid'
- padding='same'

Compare output shapes.

Explain:

- Why border information matters in crack detection.

Required Visualizations

Students must plot:

1. Training vs validation accuracy
2. Training vs validation loss
3. Overfitting comparison graph

Final Evaluation Requirement

Students must compute:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix

Explain:

- Why recall is important in crack detection?

(Hint: Missing a crack is dangerous.)